Solution: Valid Sudoku

Problem Statement

Determine if a 9x9 Sudoku board is valid. A valid Sudoku board will hold the following conditions:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. The 9 3x3 sub-boxes of the grid must also contain the digits 1-9 without repetition.

Note:

1. The Sudoku board could be partially filled, where empty cells are filled with the character '.'.
2. You need to validate only filled cells.

Example 1:

- **Input:**

```
[["5","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

- **Expected Output:** `true`
- **Justification:** This Sudoku board is valid as it adheres to the rules of no repetition in each row, each column, and each 3x3 sub-box.

Example 2:

- **Input:**

```
[["8","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

- **Expected Output:** `false`
- **Justification:** The first and fourth rows both contain the number '8', violating the Sudoku rules.

Example 3:

- **Input:**

```
[[".",".","4",".",".",".","6","3","."]
,[".",".",".",".",".",".",".",".","."]
,["5",".",".",".",".",".",".","9","."]
,[".",".",".","5","6",".",".",".","."]
,["4",".","3",".",".",".",".",".","1"]
,[".",".",".","7",".",".",".",".","."]
,[".",".",".","5",".",".",".",".","."]
,[".",".",".",".",".",".",".",".","."]
,[".",".",".",".",".",".",".",".","."]]
```

- **Expected Output:** `false`
- **Justification:** The fourth column contains the number '5' two times, violating the Sudoku rules.

Solution

- **Initialization:**
  - Create three hash sets for rows, columns, and boxes to keep track of the seen numbers.
- **Iteration:**
  - Iterate through each cell in the 9x9 board.
    - If the cell is not empty:
      - Formulate keys for the row, column, and box that include the current number and its position.
      - Check the corresponding sets for these keys.
        - If any key already exists in the sets, return `false`.
        - Otherwise, add the keys to the respective sets.
- **Final Check:**
  - If the iteration completes without finding any repetition, return `true`.

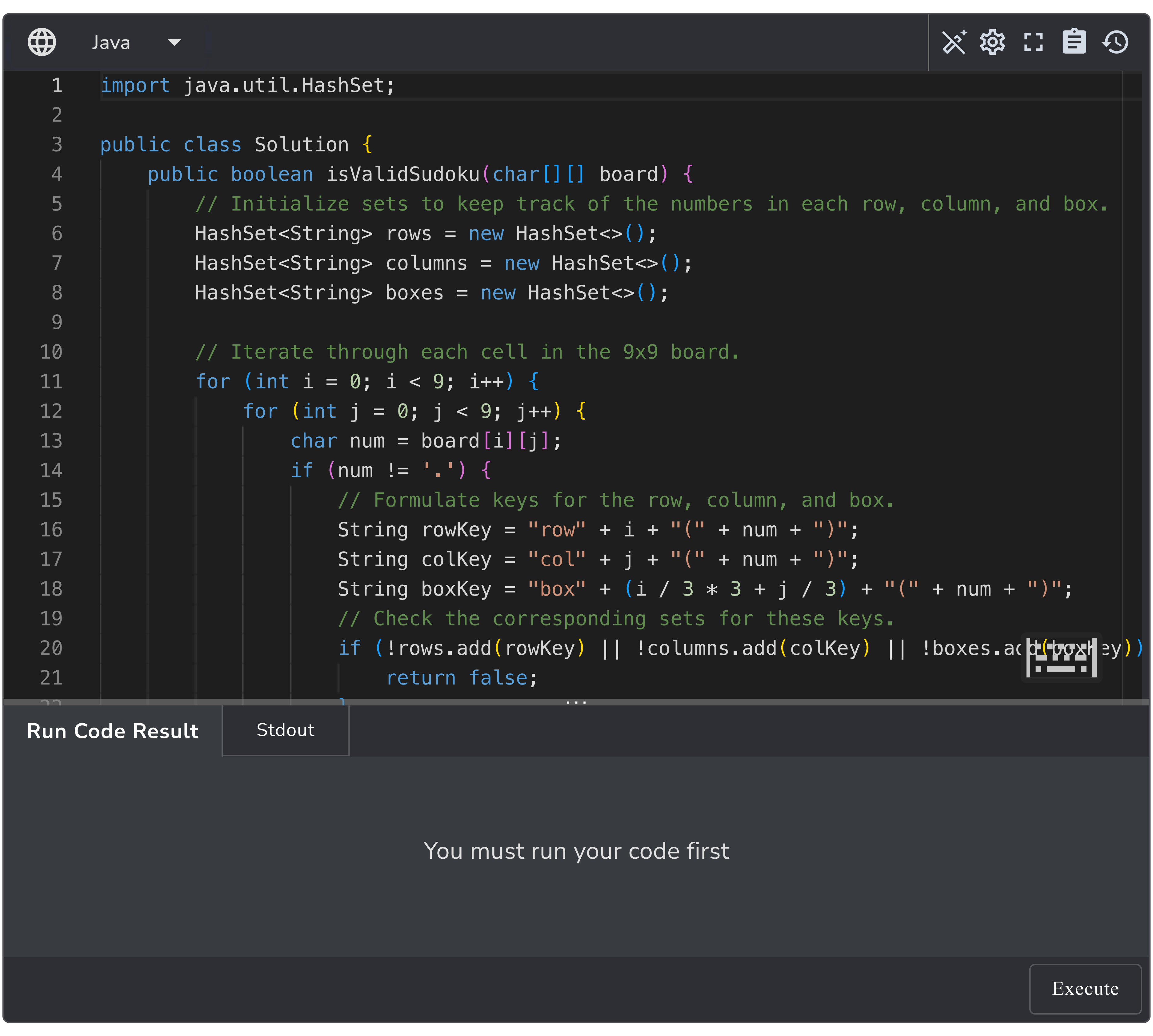
This approach works because it checks all the necessary conditions for a valid Sudoku by keeping track of the numbers in each row, column, and box using hash sets. The use of hash sets allows for efficient lookups to ensure no numbers are repeated in any row, column, or box.

Algorithm Walkthrough

Consider Example 2 from above:

- Initialize three empty hash sets for rows, columns, and boxes.
- Start iterating through each cell in the board.
  - For the first cell, which contains '8':
    - Formulate keys: row0(8), col0(8), and box0(8).
    - Since these keys are not in the sets, add them.
  - Continue this for other cells.
  - Upon reaching the first cell of the fourth row, which also contains '8':
    - Formulate keys: row3(8), col0(8), and box1(8).
    - The key col0(8) already exists in the column set, so return `false`.

Code



```java
import java.util.HashSet;

public class Solution {
    public boolean isValidSudoku(char[][] board) {
        // Initialize sets to keep track of the numbers in each row, column, and box.
        HashSet<String> rows = new HashSet<>();
        HashSet<String> columns = new HashSet<>();
        HashSet<String> boxes = new HashSet<>();

        // Iterate through each cell in the 9x9 board.
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                char num = board[i][j];
                if (num != '.') {
                    // Formulate keys for the row, column, and box.
                    String rowKey = "row" + i + "(" + num + ")";
                    String colKey = "col" + j + "(" + num + ")";
                    String boxKey = "box" + (i / 3 * 3 + j / 3) + "(" + num + ")";
                    // Check the corresponding sets for these keys.
                    if (!rows.add(rowKey) || !columns.add(colKey) || !boxes.add(boxKey))
                        return false;
```

You must run your code first

Complexity Analysis

- **Time Complexity:** O(1) or O(81), as we only iterate through the 9x9 board once.
- **Space Complexity:** O(1) or O(81), as the maximum size of our sets is 81.

← Previous
Valid Sudoku (Medium)

Next →
Product Of Array Except Self (Medium)

☐ Mark as Completed